GATE CS — Theory of Computation

Concise study notes for GATE 2027 · built from the 2016–2026 papers in this repo

Where the marks come from (average marks per paper, 2016–2026)

Topic	Marks / paper	Priority
CFG / PDA / CFL	3.6	Tier 1
Regular languages, DFA/NFA, regex	2.7	Tier 1
Turing machines, decidability	1.4	Tier 3

Theory of Computation carries about **7.7 of the 85 technical marks** in a typical paper. Topics below are listed in the order given in the notes; use the table above to decide where to spend time first.

Legend: **Trap** = common mistake · **Tip** = shortcut. For practice, use `GATE_CS_Theory_of_Computation.pdf` (past questions + new questions with solutions). These notes were written from general knowledge, not checked line by line against a textbook; verify the less common facts.

1. Regular Languages: DFA, NFA, Regular Expressions

Automata

- DFA = $(Q, \Sigma, \delta, q_0, F)$; exactly one move per state and symbol. NFA allows several moves and ε-moves; every NFA with n states converts to a DFA with at most 2^n states (subset construction). DFA, NFA, ε-NFA and regular expressions all describe the same class.
- **Minimisation:** remove unreachable states, then merge equivalent (indistinguishable) states by partition refinement (Myhill–Nerode). The minimal DFA is unique. Number of states of the minimal DFA = number of Myhill–Nerode equivalence classes.
- Standard minimal-DFA sizes: strings ending in a fixed string of length k → k + 1; strings whose k-th symbol from the right is 1 → 2^k; strings with #a ≡ r (mod k) → k; length divisible by k → k; strings containing a fixed substring of length k → k + 1; binary numbers divisible by k → k.
- To complement a DFA, make it complete and swap accepting/non-accepting states. For an NFA complement you must determinise first.
- Product construction gives union and intersection with $|Q_1|\cdot|Q_2|$ states.
- Moore and Mealy machines: output on the state (Moore) vs on the transition (Mealy); a Mealy machine outputs n symbols for n input symbols; Moore outputs n + 1.

Regular expressions

- Operators: union (+ or |), concatenation, Kleene star. Identities: (a*)* = a*, (a + b)* = (a*b*)*, a*a* = a*, ε + aa* = a*, (ab)*a = a(ba)*, ∅* = ε, rε = r, r∅ = ∅.
- Convert DFA → regex by state elimination or Arden's lemma: R = Q + RP ⇒ R = QP*.
- Count strings: number of strings of length n with some property is often a recurrence read off the DFA.

Closure and non-regular languages

Operation	Regular	CFL	Decidable	R.E.
Union	yes	yes	yes	yes
Intersection	yes	no	yes	yes
Complement	yes	no	yes	no
Concatenation	yes	yes	yes	yes
Kleene star	yes	yes	yes	yes
Reversal	yes	yes	yes	yes
Difference	yes	no	yes	no
Homomorphism	yes	yes	no (erasing)	yes
Intersection with regular	yes	yes	yes	yes

- **Pumping lemma** (to show non-regularity): for a regular L there is p such that every w ∈ L with |w| ≥ p can be written w = xyz with |xy| ≤ p, |y| ≥ 1 and $xy^iz \in L$ for all i ≥ 0. Prove non-regularity by choosing a string and showing every split fails.
- Non-regular: $\{a^nb^n\}$, {ww}, $\{ww^R\}$, {a^(n²)}, $\{a^p : p \text{ prime}\}$, {w : #a = #b}, $\{a^nb^m : n \neq m\}$ (complement trick). Regular: $\{a^nb^m : n + m \text{ even}\}$, {w : #a − #b ≡ 0 mod 3}, any finite language, $\{a^n : n \bmod 3 = 0\}$.
- Every finite language is regular. Every subset of a regular language is not necessarily regular (Σ* has non-regular subsets).
- Decidable for regular languages: membership, emptiness, finiteness, equivalence, universality, containment.

2. Context-Free Languages: CFG, PDA

- CFG $G = (V, \Sigma, P, S)$: productions $A \to \alpha$. Derivations (leftmost/rightmost), parse trees. A grammar is **ambiguous** if some string has two parse trees (equivalently two leftmost derivations). Ambiguity is undecidable; removing it may be possible by redesign (precedence levels), but some CFLs are inherently ambiguous (e.g. $\{a^i b^j c^k : i = j \text{ or } j = k\}$).
- Simplification: remove ε-productions, unit productions, useless symbols. **Chomsky normal form**: $A \to BC$ or $A \to a$ (a string of length n derives in $2n - 1$ steps). **Greibach normal form**: $A \to a\alpha$.
- **PDA**: finite control plus a stack. Accept by final state or by empty stack — equivalent in power for nondeterministic PDAs. NPDA ≡ CFG. **DPDA is strictly weaker** than NPDA: DCFLs are closed under complement but not union/intersection.
- Examples: $\{a^n b^n\}$ DCFL; $\{ww^R\}$ CFL but not DCFL; $\{a^i b^j c^k : i = j \text{ or } j = k\}$ CFL not DCFL; $\{a^n b^n c^n\}$ not CFL; $\{ww\}$ not CFL; $\{a^n b^m c^{n+m}\}$ CFL; $\{a^n b^m c^k : k = n \cdot m\}$ not CFL.
- **Pumping lemma for CFL:** $w = uvxyz$ with $|vxy| \le p$, $|vy| \ge 1$ and $uv^i xy^i z \in L$ for all $i \ge 0$. Use it to prove $\{a^n b^n c^n\}$, $\{ww\}$, $\{a^{(n^2)}\}$ are not context-free.
- Decidable for CFLs: membership (CYK, $O(n^3)$), emptiness, finiteness. **Undecidable:** equivalence, universality ($L = \Sigma^*$), ambiguity, whether $L(G_1) \cap L(G_2) = \emptyset$, whether a CFL is regular.
- Closure: union, concatenation, star, reversal, homomorphism, intersection with a regular language. Not closed: intersection, complement, difference.
- Intersection of two CFLs can be non-CFL: $\{a^n b^n c^m\} \cap \{a^m b^n c^n\} = \{a^n b^n c^n\}$.

Grammar-type hierarchy (Chomsky)

Type	Grammar	Machine
3	Regular	Finite automaton
2	Context-free	Pushdown automaton
1	Context-sensitive	Linear-bounded automaton
0	Unrestricted	Turing machine

- Regular ⊂ DCFL ⊂ CFL ⊂ CSL ⊂ Recursive ⊂ Recursively enumerable. Context-sensitive languages are decidable.

> **Tip:** To decide “CFL or not?”, ask whether one stack can check the condition. Comparing two counts is fine; comparing three, or copying a string (ww), is not.

3. Turing Machines & Decidability

- TM = finite control + infinite tape. Variants (multi-tape, nondeterministic, multi-track) have the same power as the basic TM; they differ in time (multi-tape simulation costs a quadratic slowdown).
- **Recursive (decidable)**: a TM halts on every input. **Recursively enumerable (RE)**: a TM accepts exactly the strings in L (may loop on others). L is recursive iff both L and its complement are RE. The complement of an RE non-recursive language is not RE.
- Decidable classes are closed under union, intersection, complement, concatenation, star. RE is closed under union, intersection, concatenation, star, but not complement.
- **Halting problem** (HALT): undecidable but RE. Accepts: A_TM undecidable, RE. E_TM (is L(M) empty?) undecidable, and its complement is RE. EQ_TM undecidable and not even RE.
- **Rice's theorem:** every non-trivial semantic property of the language of a TM is undecidable (e.g. is L(M) regular? finite? empty? does M accept ε?). It does not apply to syntactic properties (e.g. does M have more than 5 states?).
- **Reductions:** to prove B undecidable, reduce a known undecidable A to B ($A \leq B$). If $A \leq B$ and B is decidable then A is decidable; if $A \leq B$ and A is undecidable then B is undecidable. Direction matters.
- Undecidable problems: Post Correspondence Problem (PCP), validity in first-order logic, CFG ambiguity, CFG universality, whether two TMs are equivalent, whether a program halts, whether a TM ever writes a non-blank symbol on all inputs.
- Decidable: DFA/NFA problems, CFG membership and emptiness, halting of a TM within n steps (bounded), acceptance by a linear-bounded automaton.
- Time-bounded classes: P (polynomial time deterministic), NP (polynomial time verifiable / nondeterministic), NP-complete (in NP and every NP problem reduces to it, e.g. SAT, 3-SAT, clique, vertex cover, Hamiltonian cycle, subset sum), co-NP. $P \subseteq NP$; whether $P = NP$ is open. 2-SAT is in P; 3-SAT is NP-complete. If an NP-complete problem is in P, then $P = NP$.

> **Trap:** “Is $L(M) = \emptyset$?” is undecidable for TMs but decidable for DFAs and CFGs. Always match the problem to the machine model.

4. Exam strategy & revision checklist

- Study order: regular languages and minimal-DFA counting → CFL/non-CFL decisions and closure → decidability with Rice's theorem and reductions.
- Questions are mostly 1–2 mark MCQ/MSQ; for MSQ judge each statement separately.
- ☐ Build and minimise a DFA. ☐ Memorise the closure table. ☐ Apply the pumping lemma to $\{a^n b^n c^n\}$. ☐ Classify languages as regular / DCFL / CFL / recursive / RE. ☐ Use Rice's theorem. ☐ Redo `GATE_CS_Theory_of_Computation.pdf` past questions.